

TD 9 — Modèles économiques et open source en entreprise — Corrigé

Exercice 1 — Analyse de modèles économiques

1. Red Hat

- **Modèle principal** : Distributeur / Support + Éditeur open source (souscriptions)
- **Source de revenus** : Souscriptions annuelles incluant support 24/7, certifications, mises à jour, garanties juridiques. CA 4 Md\$/an avant rachat IBM.
- **Ce qui est gratuit** : Le code source (Fedora, CentOS Stream). Les binaires peuvent être reconstruits (Rocky Linux, AlmaLinux — d'où la controverse de 2023 sur la restriction d'accès aux sources RHEL).
- **Ce qui est payant** : Les binaires officiels RHEL, le support, les certifications, les SLA, la marque.

Attention : Red Hat n'est pas qu'un "assembleur" de composants tiers (modèle historique Red Hat Linux). C'est aussi un **éditeur** de produits développés majoritairement en interne :

- **OpenShift** (plateforme Kubernetes, avec couche propriétaire → open core de fait)
- **Ansible** (automatisation, racheté en 2015)
- **JBoss / Quarkus** (middleware Java)
- **Ceph** (stockage distribué)
- Et des contributions massives à Linux, systemd, GNOME, Podman...

Le modèle Red Hat est donc un **hybride** : distribution + édition + services + certifications.

2. GitLab

- **Modèle principal** : Open core + SaaS
- **Différence entre CE et EE** : CE (Community Edition) est open source (MIT). EE (Enterprise Edition) ajoute des fonctionnalités de sécurité avancée, compliance, gestion de portefeuille de projets, analytics. La plupart des features de gestion d'entreprise sont dans EE.
- **Ce qui attire les clients payants** : Les features de sécurité (SAST, DAST, dependency scanning), la compliance (audit, approval workflows), le support, et la version SaaS hébergée (GitLab.com) avec ses tiers payants.

3. MongoDB

- **Modèle(s) utilisé(s)** : Historiquement dual licensing (AGPL + licence commerciale), puis SSPL + SaaS (MongoDB Atlas).
- **Évolution du modèle** :
 1. AGPL v3 (open source copyleft fort) + licence commerciale pour les entreprises ne voulant pas du copyleft
 2. 2018 : passage à la SSPL (Server Side Public License) — réaction au "cloud problem"
 3. MongoDB Atlas (SaaS) devient la source de revenus principale
 4. IPO en 2017, capitalisation boursière 25 Md\$
- **Pourquoi le changement de licence (SSPL)** : AWS proposait "DocumentDB" (compatible MongoDB) sans contribuer au projet. La SSPL interdit d'offrir le logiciel comme service sans ouvrir l'intégralité de la stack. L'OSI ne reconnaît pas la SSPL comme open source.

4. WordPress/Automattic

- **Modèle principal** : SaaS + Services + écosystème
- **Relation WordPress.org / WordPress.com** :
 - WordPress.org : le projet open source (GPL v2), géré par la WordPress Foundation

- WordPress.com : le service SaaS hébergé, géré par Automattic (entreprise de Matt Mullenweg)
- Ambiguïté : Mullenweg est à la fois BDFL du projet et CEO de l'entreprise commerciale
- **Sources de revenus** : Hébergement payant (WordPress.com tiers premium), plugins commerciaux (WooCommerce, Jetpack, Akismet), services entreprise.
- **Conflit WP Engine (2024)** : Mullenweg a accusé WP Engine (hébergeur WordPress valorisé 3 Md\$, VC-funded par Silver Lake) de parasiter l'écosystème sans contribuer. Il a bloqué l'accès de WP Engine aux ressources de WordPress.org et exigé une licence trademark à 8% du CA. Cela illustre le risque du modèle single-vendor : quand la même personne contrôle le projet communautaire et l'entreprise commerciale, il n'y a pas d'arbitre neutre. La frontière entre intérêt du projet et intérêt commercial devient floue.

Question transversale

Il n'y a pas de réponse unique — chaque modèle a ses vulnérabilités face au cloud problem.

Red Hat (distributeur/support) :

- Forces : vend de la confiance, du support, des certifications — pas du code
- Mais : Rocky Linux et AlmaLinux sont des rebuilds quasi-identiques à RHEL, proposés gratuitement par tous les cloud providers. Red Hat a dû restreindre l'accès aux sources RHEL en 2023 (controverse CentOS Stream) pour protéger son modèle — signe de fragilité
- De plus, Red Hat n'est pas qu'un distributeur : il est aussi **éditeur** de produits développés en interne (OpenShift, Ansible, JBoss/Quarkus, Ceph), ce qui relève davantage de l'open core

GitLab (open core) :

- Forces : les features enterprise (sécurité, compliance) sont propriétaires — un cloud provider ne peut proposer que la version CE
- Mais : la version CE peut suffire à beaucoup d'utilisateurs, et GitHub (Microsoft) est un concurrent redoutable

MongoDB (SSPL + SaaS) :

- Le changement de licence a bloqué AWS, mais au prix de la sortie de l'écosystème open source

Conclusion attendue : Le modèle le plus résilient combine **plusieurs leviers** : code ouvert + features propriétaires (open core) + services à haute valeur ajoutée (support, certification) + SaaS avec une expérience différenciée. Aucun levier seul ne suffit.

Exercice 2 — Étude de cas : HashiCorp

1. Licence avant : MPL 2.0 (Mozilla Public License)

Licence copyleft faible (par fichier, comme la LGPL). Autorise :

- L'utilisation commerciale sans restriction
- L'intégration dans des produits propriétaires (seuls les fichiers modifiés sous MPL doivent rester sous MPL)
- La redistribution libre

C'est une licence reconnue OSI, approuvée FSF, compatible avec la GPL.

2. Nouvelle licence : BSL 1.1 (Business Source License)

Restrictions :

- **Interdit l'usage "production" en concurrence directe** avec les produits commerciaux de HashiCorp (notamment les offres cloud/SaaS)

- Les développeurs peuvent utiliser, modifier, redistribuer pour le développement, les tests, l'éducation
- **Clause de conversion** : le code devient automatiquement open source (MPL 2.0) **4 ans** après sa publication

La BSL n'est **pas** reconnue comme open source par l'OSI. C'est une licence "source-available" avec conversion différée.

3. Raison invoquée par HashiCorp

HashiCorp a invoqué la concurrence "parasitaire" des cloud providers qui proposaient des services concurrents basés sur le code Terraform/Vault/Consul sans contribuer significativement au développement. Cela correspond au "cloud problem" classique.

4. Réaction communautaire

- **Fork OpenTofu** : lancé en septembre 2023, quelques semaines après l'annonce
- Rejoint la **Linux Foundation** comme projet incubé
- Soutenu par des entreprises comme Gruntwork, Spacelift, env0 (concurrents directs de HashiCorp Cloud)
- Controverse sur la légitimité : certains estiment que les contributeurs du fork étaient surtout des concurrents commerciaux, pas la communauté open source au sens large

5. Le rachat par IBM

Le rachat par IBM (6,4 Md\$, avril 2024) intervient seulement 8 mois après le changement de licence. Plusieurs lectures :

Thèse du bait-and-switch pré-acquisition :

- Le changement de licence renforce le contrôle de HashiCorp sur son écosystème → plus attractif pour un acquéreur
- La BSL protège IBM contre la concurrence des cloud providers sur les produits HashiCorp
- Pattern déjà vu : MongoDB (SSPL → IPO), Elastic (licence propriétaire → IPO)
- Les VCs (Mayfield, GGV, IVP, qui avaient investi 350 M\$) obtiennent leur exit

Contre-argument :

- Le rachat n'était peut-être pas planifié au moment du changement de licence
- IBM rachète aussi pour la base de clients entreprise, pas seulement pour le contrôle de la licence
- HashiCorp était déjà public (IPO 2021) — le changement de licence visait peut-être à protéger le cours de bourse

Les deux lectures sont défendables. Mais le pattern VC → open source → changement de licence → exit lucratif est désormais bien documenté.

Débat

Les deux positions sont recevables. L'enseignant peut souligner :

- **Position A** est juridiquement correcte : HashiCorp détenait le copyright (via CLA) et avait le droit de changer la licence. Le financement du développement est un vrai problème.
- **Position B** est éthiquement fondée : les contributeurs externes ont travaillé sous MPL en pensant contribuer à un projet libre. Le CLA qui permet le changement de licence était le signe avant-coureur.
- La vraie question est celle du **CLA** : si HashiCorp avait utilisé un DCO (sans copyright assignment), le changement de licence aurait été impossible. Le choix du CLA est donc un indicateur de risque (cf. "Signes d'alerte" dans le cours).

Exercice 3 — Audit de conformité

1. Obligations par composant

Composant	Obligations
React (MIT)	Conserver la notice de copyright et la licence dans les distributions. Aucune autre obligation.
FFmpeg (LGPL 2.1)	Conserver copyright + licence. Linking dynamique autorisé avec du code propriétaire. Comme le code est modifié (patch audio) : obligation de fournir le code source des modifications sous LGPL 2.1.
libgit2 (GPL 2.0 + linking exception)	La “linking exception” permet d’utiliser libgit2 dans du code propriétaire sans contamination copyleft, à condition de ne pas modifier libgit2 lui-même. Conserver copyright + licence. Ici pas de modification → OK.
readline (GPL 3.0)	GPL fort, sans exception. Tout programme lié à readline (statiquement ou dynamiquement) doit être distribué sous GPL 3.0. Incompatible avec un produit propriétaire.
SQLite (Public Domain)	Aucune obligation. Le code est dans le domaine public.
OpenSSL (Apache 2.0)	Conserver copyright + licence + notice de modifications si modifié. Clause de non-utilisation des marques. Compatible avec propriétaire.

2. Composant problématique

readline (GPL 3.0) est le composant critique. La GPL 3.0 est un copyleft **fort** : tout programme qui utilise readline (même via linking dynamique) doit être distribué sous GPL 3.0. C’est **incompatible** avec un produit propriétaire.

Différence avec FFmpeg (LGPL 2.1) :

- La LGPL autorise le linking dynamique avec du code propriétaire — le code propriétaire n’a pas besoin d’être sous LGPL
- La GPL n’autorise PAS cette séparation — tout le programme est “contaminé”
- C’est la distinction fondamentale entre copyleft fort (GPL) et copyleft faible (LGPL)

Note sur libgit2 : La GPL 2.0 “pure” aurait le même problème que readline. Mais libgit2 a une **linking exception** explicite qui autorise l’usage dans du code propriétaire. C’est un pattern courant pour les bibliothèques qui veulent être GPL mais utilisables partout (similaire à la classpath exception de Java/OpenJDK).

3. Obligations supplémentaires pour le patch FFmpeg

Oui. Sans modification, l’obligation LGPL se limite à conserver les notices et permettre le re-linking (linking dynamique). Avec des modifications :

- Obligation de **fournir le code source des modifications** (le patch audio) sous LGPL 2.1
- L’utilisateur doit pouvoir reconstruire FFmpeg avec le patch et le lier au produit
- En pratique : inclure le code source modifié de FFmpeg dans la distribution ou le mettre à disposition sur demande

4. Éléments à inclure dans la distribution

- [x] Notices de copyright de tous les composants (React, lodash, FFmpeg, libgit2, OpenSSL)
- [x] Textes complets des licences (MIT, LGPL 2.1, GPL 2.0, Apache 2.0)
- [x] Code source des modifications de FFmpeg (patch audio) sous LGPL 2.1
- [x] Notice LGPL expliquant comment re-linker FFmpeg (si linking dynamique)

5. Solutions pour readline (GPL 3.0)

Plusieurs options, de la plus simple à la plus radicale :

1. **Remplacer readline** par une alternative compatible : libedit (licence BSD) est un remplacement drop-in de readline, utilisé par PostgreSQL et Python pour cette raison exacte
2. **Isoler readline** dans un processus séparé communiquant par IPC (pipe, socket) — si l'interface CLI est un programme séparé, le code propriétaire n'est pas lié à readline
3. **Passer le produit sous GPL** — rarement acceptable pour un produit commercial
4. **Supprimer la fonctionnalité CLI** qui dépend de readline — si c'est un composant non essentiel

La solution 1 (remplacement par libedit) est la plus courante dans l'industrie.

Avis de conformité

Le produit est globalement conforme, à une exception critique : readline (GPL 3.0) est incompatible avec une distribution propriétaire. Son remplacement par libedit (BSD)

est recommandé avant commercialisation. Les modifications apportées à FFmpeg (LGPL 2.1)

doivent être documentées et leur code source inclus dans la distribution. Les notices de copyright et licences de tous les composants doivent être incluses.

Étape du processus OSPO

Cet audit s'inscrit dans l'étape "**Analyse**" (validation des licences) du processus de conformité de l'OSPO, en amont de la phase "Validation" par le juridique. En cas de problème identifié (readline), il déclenche la phase de "**Remédiation**" (remplacement du composant).

Exercice 4 — Conception d'un OSPO

4.1 Mission et périmètre

Mission (exemple) :

L'OSPO de TechCorp a pour mission de permettre une utilisation responsable et stratégique de l'open source, en assurant la conformité juridique, la sécurité de la supply chain et en favorisant la contribution upstream.

Périmètre d'action :

- [x] Conformité licences — **priorité 1** vu l'incident GPL récent
- [x] Sécurité supply chain — inventaire des dépendances, vulnérabilités
- [x] Politique de contribution — formaliser ce que les développeurs font déjà "en cachette"
- [x] Relations communautaires — identifier les projets critiques pour TechCorp
- [x] Formation — sensibiliser les 500 développeurs aux licences et bonnes pratiques
- [x] Veille technologique — suivre les évolutions de l'écosystème

4.2 Organisation

Rattachement proposé : Direction technique (CTO) — mais d'autres options sont défendables.

Rattachement	Avantages	Inconvénients
CTO	Légitimité technique, accès direct aux dev, rapidité d'exécution	Peut négliger l'aspect juridique
Direction juridique	Rigueur sur la conformité, lien avec les contrats/achats	Risque de blocage, perçu comme "police", éloigné des dev
Direction produit	Alignement avec la stratégie business, priorisation claire	Peut instrumentaliser l'open source à des fins purement commerciales
Indépendant (rattaché au CEO/DG)	Transversalité, neutralité entre les directions	Difficile à justifier dans une structure de 500 personnes

Pour TechCorp, le CTO est le meilleur choix initial : l'urgence est technique (incident GPL, inventaire manquant) et les 500 développeurs sont le public principal. La direction juridique sera impliquée comme partenaire pour les audits de conformité.

Équipe initiale :

Rôle	Profil recherché	ETP
Responsable OSPO	Développeur senior avec expérience open source et sensibilité juridique	1
Ingénieur conformité	Profil DevOps/SRE, maîtrise des outils de scan (ScanCode, FOSSA)	1
Évangéliste / formateur	Développeur actif dans la communauté open source, bon communicant	0.5

4.3 Priorités année 1

1. **Inventaire complet des dépendances (SBOM)** pour tous les produits — combler le vide actuel
2. **Audit de conformité licences** — identifier et remédier les violations (comme l'incident GPL)
3. **Politique de contribution formalisée** — légitimer les contributions, définir le processus de validation
4. **Formation des développeurs** — sessions sur les licences, les risques, les bonnes pratiques
5. **Outillage CI/CD** — intégrer un scanner de licences (ScanCode, FOSSA) et un scanner de vulnérabilités (Dependabot, Snyk) dans les pipelines

4.4 KPIs proposés

KPI	Cible année 1	Méthode de mesure
% de projets avec SBOM complet	100%	Dashboard SBOM
Violations de licence identifiées et remédiées	0 violation active	Scan automatique hebdomadaire
Développeurs formés aux licences open source	80% (400/500)	Suivi des formations
Contributions upstream officielles	20+ PRs	Comptage GitHub/GitLab

Exercice 5 — Mise en place d'InnerSource

Problèmes identifiés

1. **Duplication de code** : trois bibliothèques similaires maintenues en parallèle → coût de maintenance multiplié, incohérences, bugs corrigés trois fois
2. **Silos organisationnels** : les équipes ne communiquent pas, pas de partage de connaissances, dépendance forte à chaque équipe pour “son” composant
3. **Blocages et latence** : quand l'équipe B a besoin d'une modification dans la lib de l'équipe A, elle attend (backlog) ou duplique → dette technique croissante

Comment l'InnerSource résout ces problèmes

- L'équipe B peut **soumettre une PR** directement sur la lib de l'équipe A, sans attendre
- Les libs deviennent des **composants partagés** avec une gouvernance claire (Trusted Committers)
- La **visibilité** du code et des contributions motive le partage et réduit les duplications
- Les équipes développent des **compétences transverses** en contribuant à d'autres libs

Conception d'un programme InnerSource

Projet pilote : La lib d'authentification (équipe A) — c'est la plus critique et la plus demandée par les autres équipes.

Trusted Committers identifiés :

Nom (fictif)	Équipe d'origine	Temps dédié
Alice Martin	Équipe A (auth)	20% (1 jour/semaine)
Bob Durand	Équipe C (config)	10% (0.5 jour/semaine)

Process de contribution :

1. Le contributeur ouvre une issue décrivant le besoin / la modification proposée
2. Discussion avec le Trusted Committer pour valider l'approche
3. Le contributeur soumet une PR (tests inclus, documentation mise à jour)
4. Review par le Trusted Committer, itérations si nécessaire, merge

Comment convaincre les équipes réticentes :

Objection anticipée	Réponse
“On n'a pas le temps”	Le temps dédié au Trusted Committer est officiellement alloué. Les PRs externes réduisent le backlog de l'équipe — c'est du temps gagné, pas perdu.
“C'est pas notre code”	Le Trusted Committer garde le contrôle qualité via la code review. Aucune PR n'est mergée sans son approbation. La responsabilité reste dans l'équipe.
“Ça va créer des bugs”	Les tests automatisés et la code review sont obligatoires. En pratique, les contributions InnerSource sont souvent de meilleure qualité car le contributeur sait qu'il sera reviewé par un expert du domaine.